# Claude Code 操作使用

**主讲人：** 龙建华
**适用对象：** 妙笔技术组全员
**目标：** 掌握 CLI 工具安装、核心命令交互及 IDE 插件化工作流，提升编码效率。

# 目录


- 第一阶段：基础部署与核心交互
  - 1. 安装与部署 (Installation & Deployment)
  - 2. 核心命令详解 (Core Commands)
  - 3. 上下文引用：@ 文件选择机制
- 第二阶段：进阶集成与可视化增强
  - 1. IDE 插件联动 (VS Code / IntelliJ IDEA)
  - 2. 搭配 CC Switch：可视化管理中心


---

# 第一阶段：基础部署与核心交互

**目标：** 完成环境搭建，熟悉命令行交互模式，掌握文件上下文引用。

## 1. 安装与部署 (Installation & Deployment)

在开始之前，确保本地已安装 Node.js (v18+) 或相应的运行时环境。

- **安装步骤：**

```
# 通过 npm 全局安装
npm install -g @anthropic-ai/claude-code

# 验证安装
claude --version
```

- **配置文件修改 (Host + API)：**
  - 定位配置文件（通常位于 `~/.claude/settings.json` 或通过初始化生成）。
  - **关键配置项：**
    - `host`：设置代理地址或自定义端点（如需内网穿透或特定网关）。
    - `api_key`：填入有效的 Anthropic API Key 或兼容接口的 Key。
    - `base_url`：若使用第三方转发服务，需在此处修改 API 基础地址。

- *注意：修改后需重启终端或重新初始化会话以生效。*

## 2. 核心命令详解 (Core Commands)

进入交互模式后（输入 `claude`），以下命令是日常高频操作：

| 命令 | 功能描述 | 使用场景示例 |
| --- | --- | --- |
| **`/init`** | 初始化新项目 | 在新目录下运行，生成 `.claude.md` 上下文文件，定义项目规则。 |
| **`@` + 文件名** | 引用文件上下文 | 输入 `@main.java` 或 `@src/`，将指定文件或文件夹内容直接注入当前对话上下文。 |
| **`/model`** | 切换模型版本 | 在 `Sonnet 4.5`，`Opus 4.5` 等模型间动态切换，平衡速度与智能。 |
| **`/status`** | 查看当前状态 | 检查 Token 使用情况、当前模型、上下文窗口剩余量。 |
| **`/compact`** | 压缩上下文 | 自动摘要对话历史，剔除冗余试错过程，保留核心决策与 TODO，节省 Token 并提升响应速度。 |
| **`Shift + Tab`** | 切换运行模式 | 在**编码模式（Accept Edits ）**（直接执行/写代码）与**计划模式（Plan Mode）**（输出方案/步骤分析）之间无缝切换，控制 AI 是先思考还是先行动。 |
| **`/resume`** | 恢复中断的会话 | 当网络波动或意外退出时，找回之前的对话历史和上下文。 |
| **`/mcp`** | 管理 MCP 服务器 | 连接外部数据源或工具（如数据库、文件系统），扩展 AI 能力边界。 |
| **`/skills`** | 加载技能包 | 调用预设的复杂任务模板（如“重构代码”、“编写单元测试”）。 |
| **`/memory`** | 管理长期记忆 | 项目提示词/全局提示词。查看或清除 AI 记住的项目特定偏好和历史信息。 |

## 3. 上下文引用：@ 文件选择机制

这是 CLI 模式下最高效的功能之一，无需复制粘贴代码。

- **用法：** 在输入提示词时，输入 `@` 符号。
- **功能：**
  - `@filename.py`：引用具体文件内容。
  - `@folder/src`：引用整个文件夹的代码结构。
  - `@git-log`：引用最近的提交记录。

- **示例：**

> “请分析 @main.java 和 @utils/helper.java 的逻辑，并在 @tests 目录下生成对应的单元测试文件。”

---

# 第二阶段：进阶集成与可视化增强

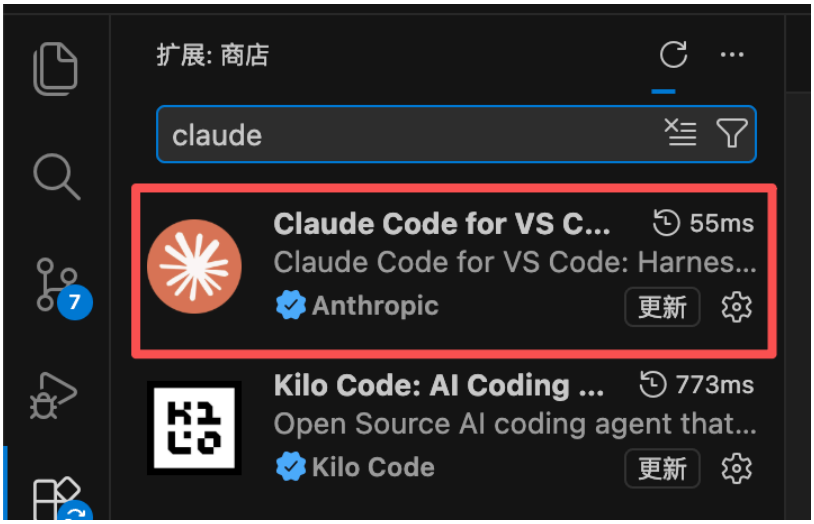
**目标：** 打通 IDE 工作流，实现图形化监控与多模型管理。

## 1. IDE 插件联动 (VS Code / IntelliJ IDEA)

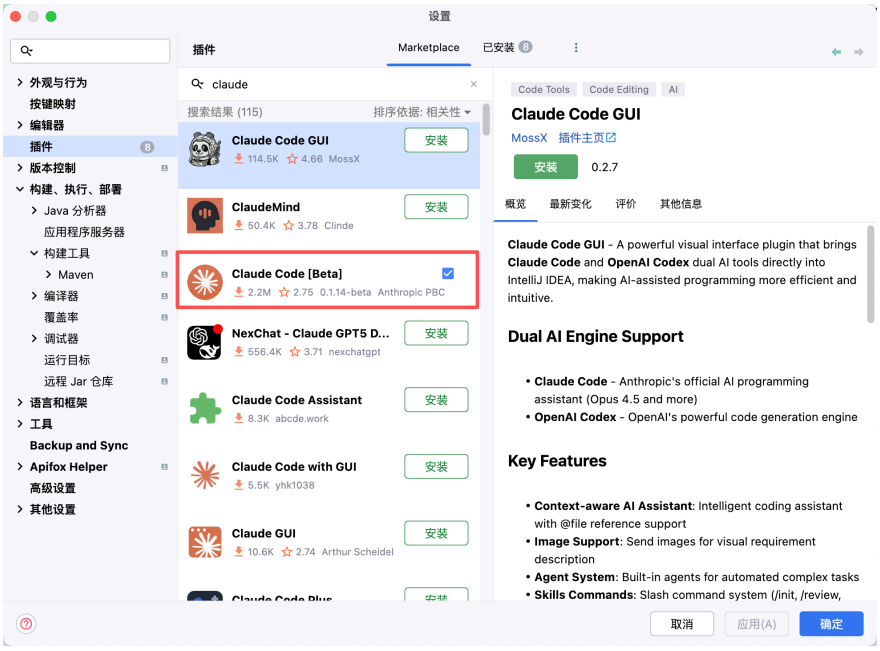
将 CLI 的强大能力嵌入开发者最熟悉的编辑器中。

- **安装步骤：**
  1. 打开 VS Code 或 IntelliJ IDEA 的 **插件市场 (Extensions/Plugins)**。

VS Code 如图



IntelliJ IDEA



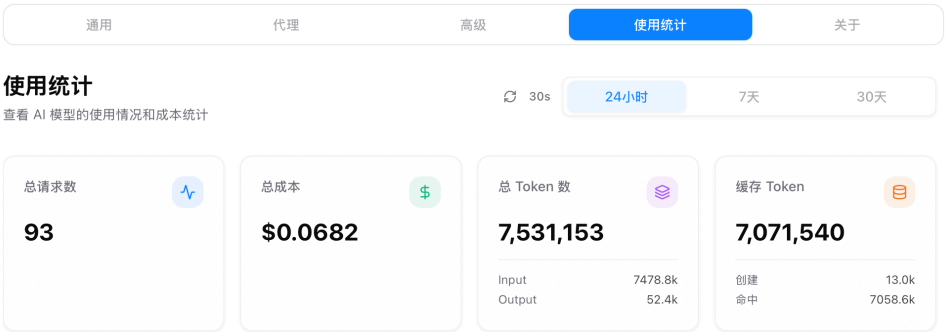

```
2. 搜索 `Claude` 或 `Anthropic` 官方/社区插件并安装。
3. 输入/ide 联动ide插件。
```

- **核心效果：文件勾选联动**
  - **操作方式：** 在 IDE 侧边栏直接勾选多个文件（支持 Ctrl/Cmd 多选）。
  - **联动机制：** 勾选的文件自动作为上下文注入到 CLI 对话框中，无需手动输入 `@` 路径。
  - **优势：** 极大减少路径输入错误，直观管理上下文范围，实现“所见即所得”的编码辅助。

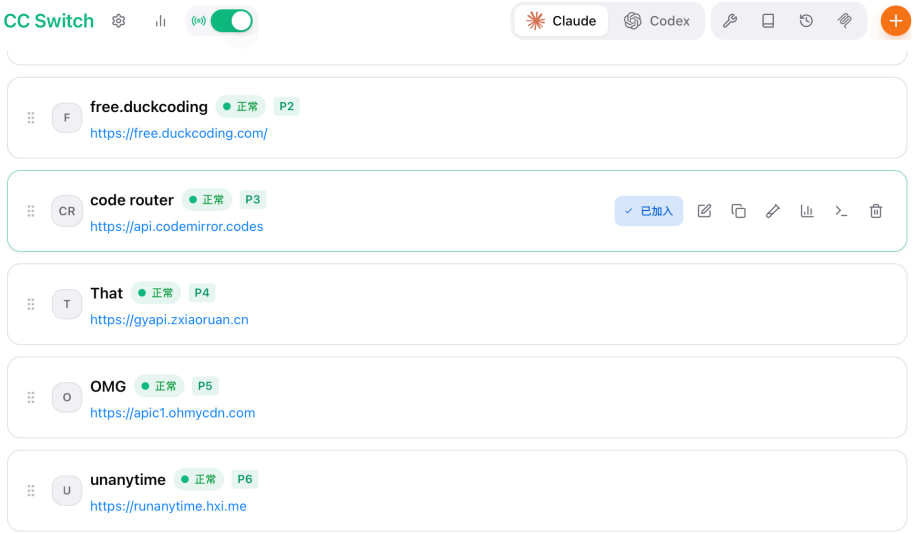
## 2. 搭配 CC Switch：可视化管理中心

项目地址： `https://github.com/farion1231/cc-switch/releases`
引入 `ccswitch` (或同类管理工具) 解决多端点、多模型及成本监控痛点。

- **主要功能模块：**
  1. **Token 消耗可视化：**
     - 实时图表展示每次请求的 Input/Output Token 数。
     - 估算美元成本，帮助团队控制预算。

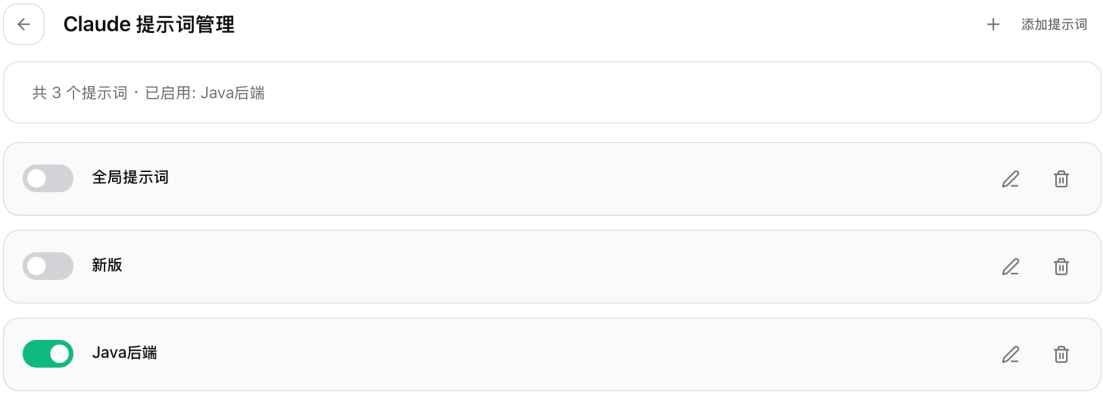



  2. **动态切换端点 (Endpoint Switching)：**
     - 一键在官方 API、私有化部署实例、第三方中转服务之间切换。无需再改配置文件重启cli客户端。
     - 支持故障自动转移配置。




  3. **全局提示词设置 (System Prompt UI)：**
     - 提供图形化界面编辑全局 System Prompt，无需修改 JSON 配置文件。

- 支持为不同项目预设不同的“角色设定”。




4. **多 CLI 工具支持：**
   - 统一入口管理多种后端模型：
     - **Claude** (Anthropic)
     - **Codex**（OpenAi）
     - **Gemini** (Google)
     - **OpenCLaw** (智能体)
   - 允许在一次会话中对比不同模型的输出结果。